Hochschule für Technik
und Wirtschaft Berlin
University of Applied Sciences

Projektdokumentation

Website für einen Förderverein

Gruppe 3

Anhalt, Johann Jacob
Bleyer, Raphael
Bommer, Max Michael
Hoang, Nguyen Viet
Tran, Tommy

Modul IT-Infrastruktur
Sommersemester 2026

Inhaltsverzeichnis

1. Einleitung & Projektkontext 4
1.1 Aufgabenstellung 4
1.2 Zielsetzung der Infrastruktur 4
1.3 Aufbau dieses Dokuments 4
1.4 Arbeitsverteilung 5
2. Architektur-Überblick 5
2.1 Stack 5
2.2 Netzwerksegmentierung 6
2.3 Datenfluss eines Requests 7
2.4 Diagramme 8
3. Inbetriebnahme 10
3.1 Voraussetzungen 10
3.2 Secrets anlegen 10
3.3 Start 10
3.4 Verifikation 11
3.5 Stoppen 11
4. Komponenten im Detail 11
4.1 Reverse Proxy (`reverse-proxy`) 11
4.2 Frontend (`frontend`, `frontend-2`) 12
4.3 Load Balancer (`load-balancer`) 12
4.4 Backend (`backend-1`, `backend-2`) 13
4.5 Datenbank (`database`) 14
4.6 Monitoring-Stack 14
5. API-Referenz 15
5.1 GET /health 15
5.2 POST /api/mitglieder 15
5.3 POST /api/spenden 15
5.4 GET /metrics 16
6. Verfügbarkeit & Skalierung 16
6.1 Ausfall eines Frontend-Webservers ohne Verfügbarkeitsverlust 16
6.2 Backend-Ausfall im Monitoring sichtbar 16
6.3 Frontend zeigt bei Backend-Ausfall passende Fehlermeldung 16
6.4 Datenbank-Ausfall im Monitoring sichtbar 16
6.5 Frontend zeigt bei DB-Ausfall passende Fehlermeldung 16
6.7 Automatischer Container-Restart 17
7. Monitoring & Alerting 17
7.1 Prometheus-Targets 17
Der Blackbox Exporter ist bewusst in internal-net und public-net eingebunden – internal-net damit Prometheus ihn scrapen kann, public-net damit er aktiv HTTP-Probes gegen die öffentlich erreichbaren Services durchführen kann 17
7.2 Grafana-Dashboards 17
7.3 Alert-Regeln 18
7.4 Verifikation eines Alerts 19

8. Sicherheit.....20
8.1 Netzwerkisolation.....20
8.2 TLS.....20
8.3 Docker Secrets (statt ENV-Variablen).....20
8.4 Read-Only-Dateisysteme & Capabilities.....21
8.5 Web Application Firewall (ModSecurity / OWASP CRS).....22
8.6 Sicherheitsscan (Trivy).....23
8.7 Exponierte Ports.....24
9. Backup & Recovery.....25
9.1 Aufbau.....25
9.2 Restore.....25
10. Lasttests.....26
10.1 Setup.....26
10.2 Szenarien & Ergebnisse.....26
10.3 Nötige Anpassungen am Server-Code.....27
10.4 Methodische Anmerkung.....27
10.5 Kurzbewertung.....28
11. Bemerkungen.....28
11.1 Bekannte Limitationen.....28
11.2 Quellen & Werkzeuge.....28

1. Einleitung & Projektkontext

1.1 Aufgabenstellung

Das Projekt verlangt eine containerisierte IT-Infrastruktur für die Website eines fiktiven Fördervereins, die

- über `docker compose up` reproduzierbar startbar ist,
- die in [`bewertungskriterien.md`](bewertungskriterien.md) aufgelisteten Lasttest-Szenarien aushält,
- Ausfälle einzelner Komponenten kompensiert oder zumindest sichtbar macht,
- gängige Sicherheitsmaßnahmen umsetzt (Netzsegmentierung, TLS, Secrets, Capability-Reduktion, WAF, CVE-Freiheit),
- ein automatisches Datenbank-Backup besitzt.

Die fachliche Anwendung ist bewusst klein gehalten: eine statische Vereins-Website mit zwei Formular-Endpunkten (Mitgliedsantrag, Spende). Den Großteil der Arbeit macht das Drumherum – Replikation, Reverse-Proxy mit WAF, Monitoring-Stack, Backups und die Härtung der Container.

1.2 Zielsetzung der Infrastruktur

Drei Eigenschaften standen im Vordergrund:

Eigenschaft	Bedeutung in diesem Projekt
Verlässlichkeit unter Last	Endpunkte halten 1000 parallele Anfragen ohne Server-Fehler aus (siehe Lasttests).
Erkennbarkeit von Problemen	Jeder Ausfall – Container, DB, HTTP-Probe, hohe CPU/RAM – wird kontinuierlich gemessen und löst nach einer konfigurierten Pending-Period einen Alert aus, der per Webhook sichtbar gemacht wird.
Reduzierte Angriffsfläche	Read-only-Dateisysteme, cap_drop: ALL, Docker-Secrets statt Env-Variablen, WAF vor jedem Request, 0 HIGH/CRITICAL CVEs in den eigenen Images.

1.3 Aufbau dieses Dokuments

Kapitel 2 gibt den Überblick (Diagramm, Stack, Netze), Kapitel 3 zeigt die Inbetriebnahme. Kapitel 4–9 beschreiben jede Komponente detailliert mit ihrer Begründung. Kapitel 10 dokumentiert die Lasttests. Kapitel 11 ist eine Mapping-Tabelle Bewertungskriterium → Umsetzung → Beleg-Stelle in dieser Doku – sie ist das schnellste Werkzeug zur Prüfung der Vollständigkeit.

1.4 Arbeitsverteilung

- Anwendungsfall, Konzept (Max)
- Frontend, Firewall (Jacob)
- Netzwerkkonfiguration (Tommy und Raphael)
- Monitoring (Viet)
- Reverse Proxy + Firewall (Jacob)
- SSL (Max)
- Docker Secrets (Raphael)
- DB-Backup (Tommy)
- Lasttests (Max)

2. Architektur-Überblick

2.1 Stack

Schicht	Technologie	Bild / Quelle
Reverse Proxy + WAF	Nginx + ModSecurity (OWASP CRS)	`owasp/modsecurity-crs:nginx-alpine`
Frontend (statisch)	Nginx	eigenes Image, Base `nginx:alpine`
Load Balancer	Nginx (Round-Robin)	`nginx:alpine`
Backend	Node.js 20 / Express	eigenes Image, Base `node:20-alpine`
Datenbank	PostgreSQL 17	eigenes Image, Base `postgres:17-alpine` (mit neu kompiliertem `gosu`)
Metriken	Prometheus	`prom/prometheus:latest`
Dashboards & Alerts	Grafana	`grafana/grafana:latest`
Container-Metriken	cAdvisor	`gcr.io/cadvisor/cadvisor:latest`
Nginx-Metriken	nginx-prometheus-exporter	`nginx/nginx-prometheus-exporter:latest`

Postgres-Metriken	postgres-exporter	`prometheuscommunity/postgres-exporter:latest`
HTTP-Probes	blackbox-exporter	`prom/blackbox-exporter:latest`
Alert-Empfänger	http-echo-Webhook	`mendhak/http-https-echo:latest`
DB-Backup	`pg_dump` in Loop-Skript	`postgres:17-alpine` (Sidecar)

Insgesamt **14 Services**, davon 4 eigene Images (`backend-1`, `backend-2`, `frontend` + `frontend-2`, `database`).

2.2 Netzwerksegmentierung

Drei Docker-Netze trennen die Schichten:

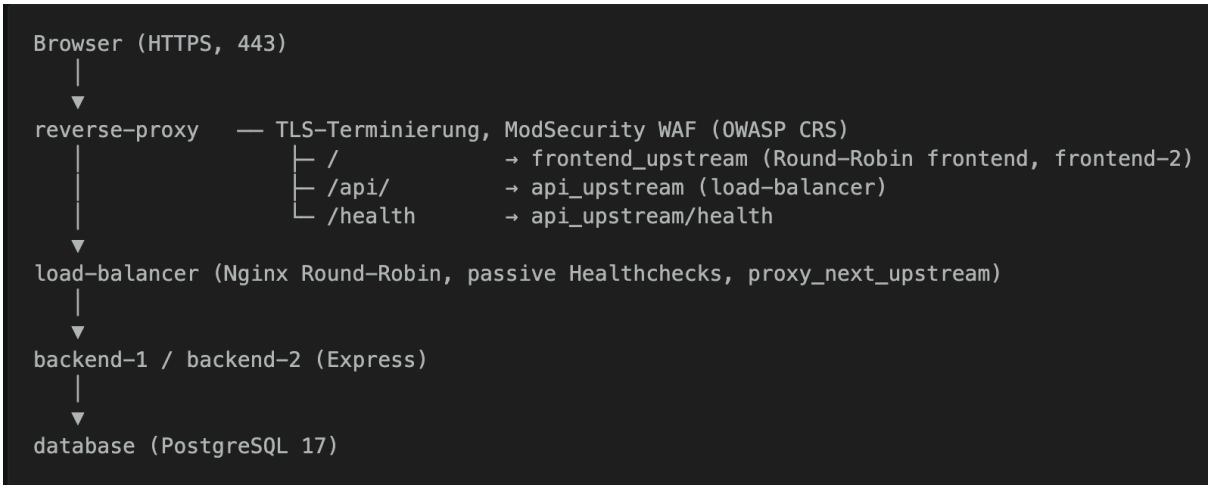
Netzwerk	**`internal`**	**Mitglieder**
`public-net`	nein	`reverse-proxy`, `load-balancer`, `frontend`, `frontend-2`, `nginx-exporter`, `blackbox-exporter`
`internal-net`	**ja**	`load-balancer`, `backend-1`, `backend-2`, `database`, `db-backup`, `postgres-exporter`, `prometheus`, `blackbox-exporter`
`monitoring-net`	nein	`prometheus`, `grafana`, `cadvisor`, `nginx-exporter`, `postgres-exporter`, `webhook-logger`

Begründung der drei Netze:

- **`internal-net` ist `internal: true`** — Container in diesem Netz haben *keine* eigene Default-Route ins Internet. Selbst wenn ein Backend kompromittiert würde, könnte es nicht direkt nach außen telefonieren. Das ist die wichtigste Single-Netz-Maßnahme dieses Projekts.
- **Trennung Public / Internal** — Nur Reverse-Proxy und Load-Balancer haben einen Fuß in beiden Welten. Der Load-Balancer ist die einzige Brücke zwischen Public-Traffic und Backend-Pool; Backend und Datenbank sind aus dem Internet nicht direkt erreichbar.
- **Monitoring-Netz separat** — Grafana muss Prometheus erreichen, Prometheus muss alle Exporter erreichen, aber nichts davon braucht Internet. Dass `prometheus`

auch im `internal-net` hängt, ist nötig, damit es Backend und postgres-exporter scrapen kann.
- Dass `blackbox-exporter` zusätzlich in `public-net` hängt, ist bewusst: Er muss aktiv HTTP-Probes gegen die öffentlich erreichbaren Services (Frontend, Load Balancer, Reverse Proxy) durchführen. Ohne `public-net` käme er an diese Ziele nicht heran. `internal-net` braucht er, damit Prometheus ihn scrapen kann.

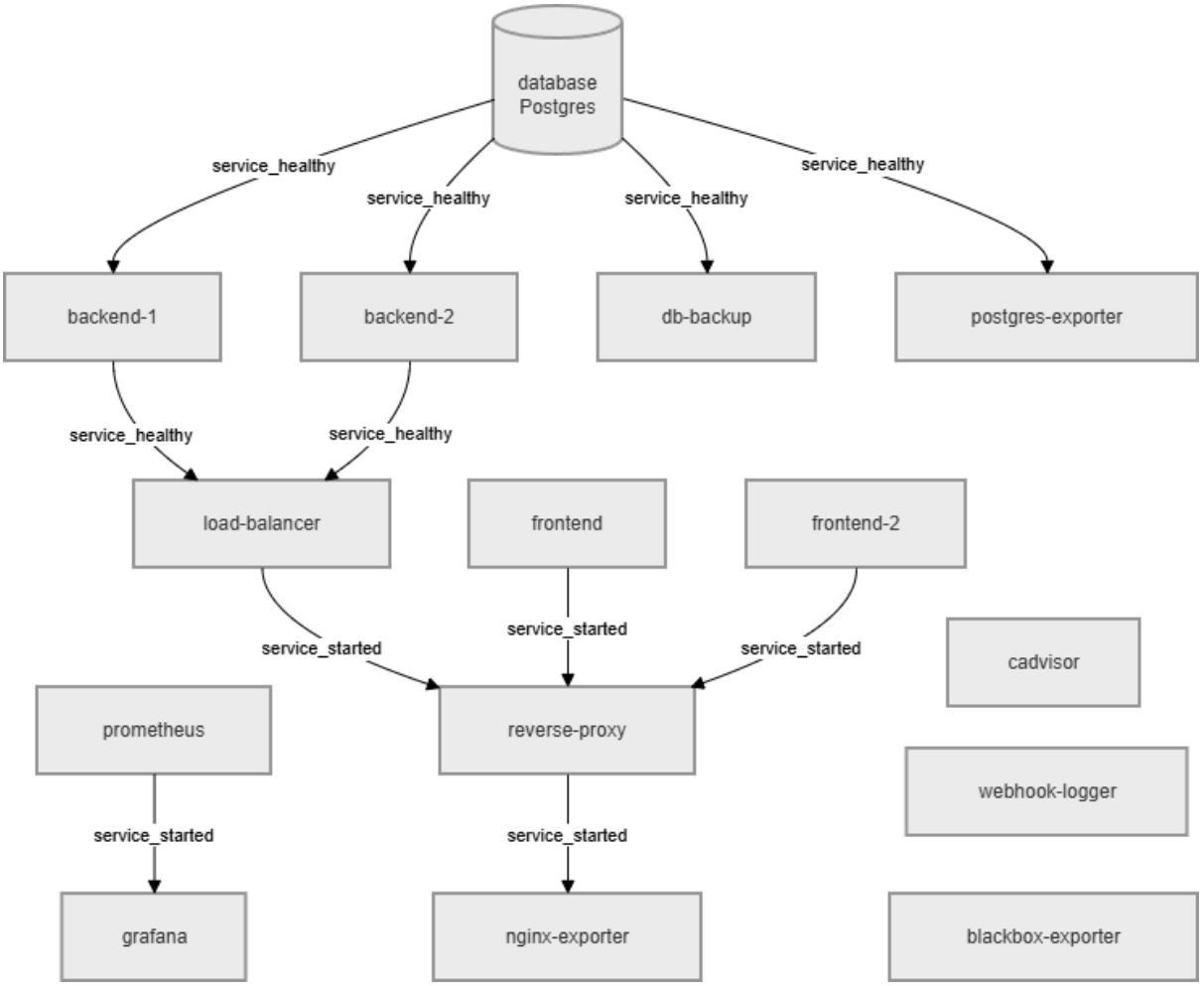
2.3 Datenfluss eines Requests



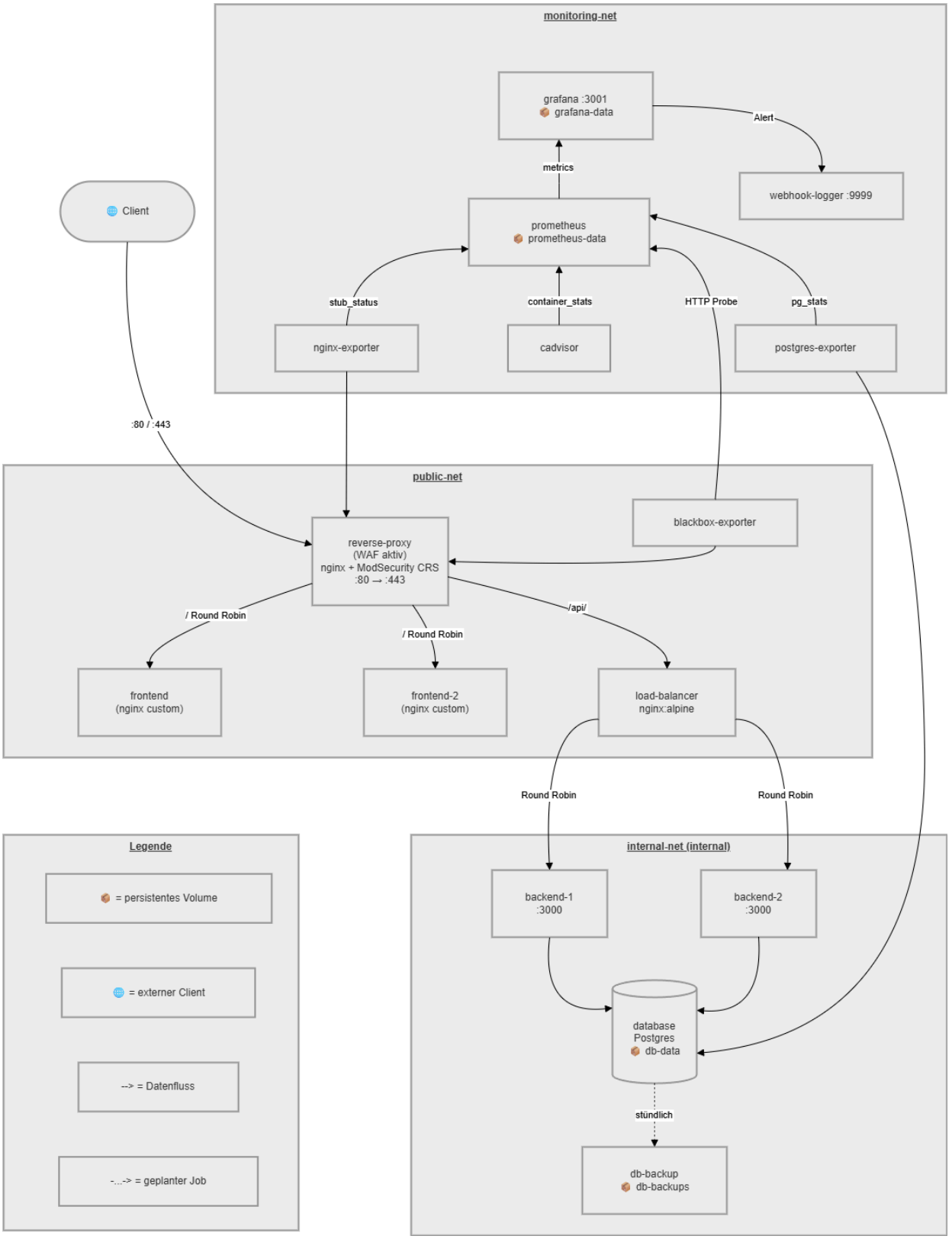
Parallel dazu scraped Prometheus alle 15 s die Exporter und das `/metrics`-Endpoint der Backends; Grafana liest aus Prometheus und feuert Alerts gegen den `webhook-logger`.

2.4 Diagramme

Dependency-Diagramm



Netzwerkdiagramm:



3. Inbetriebnahme

Eine genaue Beschreibung der Inbetriebnahme ist unter max/README.md zu finden. Dieses Kapitel ergänzt sie um die Hintergrundbegründungen.

3.1 Voraussetzungen

- Docker Desktop ≥ aktuelle Version, Docker Compose V2.
- **Docker-Desktop-Memory mindestens 6 GB** (Settings → Resources → Memory). Die 1000-VU-Lasttests brauchen sonst mehr RAM als die Docker-VM hergibt; k6 wird OOM-gekillt.
- Keine weiteren Host-Installationen — k6, Trivy, Postgres-Client laufen alle als Docker-Container.

3.2 Secrets anlegen

Vor dem ersten docker compose up:

```
mkdir secrets
[System.IO.File]::WriteAllText("secrets\postgres_password.txt
",       "DeinPostgresPasswort",
[System.Text.Encoding]::ASCII)
[System.IO.File]::WriteAllText("secrets\grafana_admin_passwor
d.txt",  "DeinGrafanaPasswort",
[System.Text.Encoding]::ASCII)
```

Der Ordner “secrets” ist in der .gitignore abgelegt und wir folglich auch nicht auf github gespeichert. So wird ein versehentliches Abfließen der Secrets gewährleistet.

3.3 Start

```
cd max
docker compose up --build -d
```

Beim ersten Start passiert intern:

1. Alle Images werden gebaut (Backend, Frontend, Database — jeweils Multi-Stage).
2. `database` startet und führt `db/init.sql` aus (legt Schema, Indizes, Trigger an).
3. Sobald `pg_isready` erfolgreich antwortet, startet `backend-1/2` (via `depends_on: condition: service_healthy`).
4. Sobald die Backends ihren `/health`-Endpunkt erfolgreich beantworten, startet der `load-balancer`.
5. Anschließend startet der `reverse-proxy` (WAF + TLS).
6. Monitoring-Stack startet parallel (hängt nur an Prometheus + Datenbank, nicht an der App-Kette).

Nach ~30 s sind alle Container `healthy`.

3.4 Verifikation

```
docker compose ps # alle Services sollten "healthy" zeigen
curl -k https://localhost/health
 # {"status":"healthy","db":"connected",...}
curl -kI https://localhost/ # 200, statische Website
open https://localhost/ # macOS: Browser
```

Der Browser zeigt eine Zertifikatswarnung (self-signed) an, da das SSL-Zertifikat vom Host selber erstellt wurde. Diese Warnung kann übersprungen werden.

3.5 Stoppen

```
docker compose down        # behält Volumes
docker compose down -v     # löscht db-data, prometheus-data,
grafana-data, db-backups
```

4. Komponenten im Detail

4.1 Reverse Proxy (`reverse-proxy`)

- **Image:** `owasp/modsecurity-crs:nginx-alpine`
- **Aufgaben:** TLS-Terminierung · WAF · HTTP→HTTPS-Redirect · Routing nach Pfad
- **Config:** `max/nginx/reverse-proxy.conf`

Architektonisch der einzige Container, der Ports an den Host bindet (`80:80`, `443:443`). Alle anderen Services sind nur containerintern erreichbar — das erfüllt das Kriterium „keine unnötigen Ports exponiert" (Ausnahmen: `grafana` auf 3001, `webhook-logger` auf 9999 für Demo-Zwecke).

Wichtige Entscheidungen:

- **TLS nur am Reverse-Proxy**, intern HTTP. Bei einem Single-Host-Compose-Setup gibt es keinen Pfad, auf dem ein Angreifer den internen Traffic mitlesen könnte, ohne bereits Container-Zugriff zu haben. Interne mTLS wäre Selbstzweck.
- **OWASP ModSecurity**. Als Regelwerk wurde das OWASP Core Rule Set (CRS) gewählt, da es der etablierte Industrie-Standard ist, um gezielt die OWASP Top 10 Schwachstellen (wie SQLi und XSS) abzuwehren, ohne eigene Regeln schreiben zu müssen.
- **WAF nur auf 443.** Auf Port 80 läuft nur der `301`-Redirect und ein Health-Check; dort ist ModSecurity überflüssig und würde die Redirect-Latenz erhöhen.

- **`client_max_body_size 6m`** — sonst lehnt Nginx 5-MB-Lasttest-Bodies mit HTTP 413 ab, bevor das Backend sie sieht (Default ist 1 MB). Die 6 statt 5 MB sind Puffer für Form-Boundaries und Header.
- **PID-Datei nach `/tmp/nginx.pid`.** Das Image braucht beim Start einen schreibbaren Pfad für die PID; per Default `/run/nginx.pid`, das aber im Read-only-Mount unter `tmpfs:` separat behandelt werden müsste. Verschieben in das ohnehin schon getmpfste `/tmp` ist die einfachere Lösung.
- **Template-Mount statt fixer Config.** Das Image rendert beim Start `/etc/nginx/templates/*.template` über `envsubst` nach `/etc/nginx/`. Direktes Überschreiben von `/etc/nginx/nginx.conf` würde mit dem Container-Init-Script kollidieren.

Healthcheck: `wget -qO- http://localhost:80/rp-health` — separater Endpunkt, der direkt aus Nginx antwortet (ohne Proxy-Weiterleitung). Wäre `/` der Health-Check, würde ein Backend-Ausfall den Reverse-Proxy fälschlich als ungesund markieren.

4.2 Frontend (`frontend`, `frontend-2`)

- **Image:** eigenes, Base `nginx:alpine`
- **Inhalt:** statisches HTML/CSS aus `max/frontend/html/` (`index.html`, `mitglied-werden.html`, `spenden.html`, `style.css`, Logo)
- **Replikation:** 2 Instanzen, vom Reverse-Proxy via Round-Robin angesprochen.
- **Begründung für zwei statt einer Instanz:** Erfüllt das Kriterium „Ausfall eines Webservers für das Frontend beeinflusst nicht die Verfügbarkeit". Bei zwei Instanzen kann eine entfernt werden (`docker compose stop frontend`) und der Reverse-Proxy leitet automatisch alle Requests an `frontend-2` weiter (passive Healthchecks via `max_fails=3 fail_timeout=10s`).

4.3 Load Balancer (`load-balancer`)

- **Image:** `nginx:alpine`
- **Aufgaben:** Round-Robin-Verteilung auf `backend-1` und `backend-2`, Fail-Fast-Timeouts, automatisches Re-Routing bei Backend-Fehlern.
- **Config:** `max/nginx/load-balancer.conf`

```
upstream backends {
    server backend-1:3000 max_fails=3 fail_timeout=30s;
    server backend-2:3000 max_fails=3 fail_timeout=30s;
}
```

Wichtige Direktiven:

Direktive	Wert	Begründung
`max_fails=3 fail_timeout=30s`	passive HC	Fällt ein Backend 3× in 30 s aus, wird es 30 s aus der Rotation genommen — verhindert, dass jeder Request während eines Crashes erst in einen Timeout läuft.
`proxy_connect_timeout 5s`	Fail-Fast	Standard ist 60 s. Bei einem hängenden Backend würden Clients sonst 60 s auf den ersten Byte-Output warten.
`proxy_read_timeout 10s`	Fail-Fast	Begrenzt die Wartezeit auf eine Antwort.
`proxy_next_upstream error timeout http_502 http_503 http_504`	aktive Failover	Bei genau diesen Fehlern wird der Request automatisch am nächsten Backend nochmal versucht — der Client merkt vom Ausfall einer Instanz nichts.

Warum Round-Robin und nicht `least_conn`?
Beide Backends sind identisch, die Requests sehr kurzlebig (DB-Insert oder `SELECT 1`). Round-Robin ist deterministisch und im Verhalten leichter zu debuggen.

4.4 Backend (`backend-1`, `backend-2`)

- **Image:** eigenes, Base `node:20-alpine` (Multi-Stage)
- **Code:** `max/backend/server.js` (~130 Zeilen)
- **Dependencies:** `express`, `pg`, `prom-client`

Verantwortlichkeiten:

1. `/health` → DB-Ping (`SELECT 1`) + Process-Uptime.
2. `/metrics` → Prometheus-Format (Default-Node-Metriken + `http_request_duration_seconds`-Histogram).
3. `POST /api/mitglieder` → INSERT in `mitglieder`, Konflikt-Handling für duplikate E-Mails (PG-Errorcode `23505`).
4. `POST /api/spenden` → INSERT in `spenden`, Validierung `betrag_cent` als positive Ganzzahl.

Wichtige Entscheidungen:

- **Passwort via Docker Secret, nicht ENV:** `POSTGRES_PASSWORD_FILE=/run/secrets/postgres_password` wird gelesen via `fs.readFileSync(...)` beim Start. `docker inspect` zeigt nur den Dateipfad, nie das Passwort.
- **`express.json({ limit: '6mb' })`:** Default wäre 100 KB. Das Lasttest-Szenario 12 schickt 5 MB Bodies; ohne diese Anhebung würde Express mit HTTP 413 antworten, der Test käme nie an die DB.
- **Connection-Pool aus `pg`:** Default ist max 10 parallele Connections. Pro Backend-Instanz × 2 Backends = max. 20 parallele DB-Connections. Bei 1000 parallelen Lasttest-VUs werden die Pool-Slots geteilt — die DB sieht also nie 1000 gleichzeitige Verbindungen, sondern 20.
- **Eigenes Histogram statt nur Default-Metriken:** Erlaubt Latenz-Panels in Grafana per Pfad/Status-Code aufzuschlüsseln. Default-Metriken liefern nur Node-interne Werte (Heap, GC, etc.).

Healthcheck: `wget -qO- http://localhost:3000/health` — `/health` selbst pingt die DB, der Healthcheck deckt also die End-to-End-Erreichbarkeit ab.

4.5 Datenbank (`database`)

- **Image:** eigenes, Base `postgres:17-alpine`, **mit neu kompiliertem `gosu`-Binary** (siehe § 8.6 Trivy).
- **Init:** `max/db/init.sql` wird beim ersten Start automatisch ausgeführt (Convention `/docker-entrypoint-initdb.d/`).
- **Schema:** `datenbankschema.md`
- **Tabellen:** `mitglieder` (Mitgliedsanträge), `spenden` (Spendeneingänge mit FK auf `mitglieder.id`, NULL erlaubt für anonyme Spenden).

Persistenz: Volume `db-data` auf `/var/lib/postgresql/data`.

4.6 Monitoring-Stack

Einzelheiten zum Monitoring sind unter *7. Monitoring und Alerting* aufgeführt. Hier wird nur eine Übersicht der Komponenten dargestellt.

Service	Aufgabe

`prometheus`	Scraped alle 15 s die Targets aus `max/monitoring/prometheus.yml`.
`grafana`	Liest Prometheus, zeigt Dashboards, führt Alert-Rules aus.
`cadvisor`	Container-Metriken (CPU, RAM, Netzwerk, Disk pro Container).
`nginx-exporter`	Übersetzt `stub_status` des Reverse-Proxy nach Prometheus-Format.
`postgres-exporter`	Übersetzt Postgres-`SHOW`/`pg_stat_*` nach Prometheus-Format (Connections, Query-Counts, etc.).
`blackbox-exporter`	HTTP-Probes (200/Connect-Time) gegen Frontend, Load-Balancer, Reverse-Proxy.
`webhook-logger`	Loggt eingehende Alert-Webhooks von Grafana zur Verifikation (Demo-Empfänger).

5. API-Referenz

Alle Endpunkte sind unter `https://localhost/api/` erreichbar (durch Reverse-Proxy mit WAF und TLS).

5.1 GET /health

Prüft Backend + DB-Verbindung. Wird sowohl von außen (Browser/Curl), als auch von Docker-Healthchecks und Prometheus-Blackbox-Probes verwendet.

- Status 200: Backend funktionsfähig und DB antwortet
- Status 503: DB-Connection schlägt fehl.

5.2 POST /api/mitglieder

Legt einen Mitgliedsantrag an. Pflichtfelder: `vorname`, `nachname`, `email`.

- Status 201: Antrag gespeichert. Antwort: `{ "id": <int> }`.
- Status 400: Pflichtfeld fehlt.
- Status 409: E-Mail bereits registriert (`UNIQUE`-Constraint).
- Status 500: Interner Fehler (DB nicht erreichbar etc.).

5.3 POST /api/spenden

Speichert eine Spende. Pflichtfeld: betrag_cent (positive Ganzzahl).

- Status 201: Antrag gespeichert. Antwort: `{ "id": <int> }`.
- Status 400: `betrag_cent` fehlt oder ist keine positive Ganzzahl.
- Status 500: Interner Fehler (DB nicht erreichbar etc.).

5.4 GET /metrics

Liefert Prometheus-Metriken im Text-Format. Wird nicht öffentlich ausgewiesen, ist aber containerintern auf `backend-1:3000/metrics` und `backend-2:3000/metrics` erreichbar — Prometheus scraped beide.

6. Verfügbarkeit & Skalierung

Die Bewertungskriterien (Block „Verfügbarkeit", 6 Punkte) verlangen sechs konkrete Fähigkeiten. Jede wird hier mit ihrer Umsetzung dokumentiert.

6.1 Ausfall eines Frontend-Webservers ohne Verfügbarkeitsverlust

Umsetzung: Zwei identische Frontend-Container (`frontend`, `frontend-2`), beide unter dem `frontend_upstream`-Block des Reverse-Proxy in `max/nginx/reverse-proxy.conf` mit `max_fails=3 fail_timeout=10s`. Fällt einer aus, antwortet der andere — der Reverse-Proxy macht passives Failover.

6.2 Backend-Ausfall im Monitoring sichtbar

Umsetzung: Grafana-Alert-Regel **„Backend ausgefallen"** (Folder `Backend`, Interval 10 s) auf der PromQL-Query `up{job="backend"}`. Bei Wert `< 1` für ≥ 20 s feuert der Alert an den `webhook-logger`.

Konfiguration: `max/monitoring/grafana/provisioning/alerting/rules.yaml`.

6.3 Frontend zeigt bei Backend-Ausfall passende Fehlermeldung

Umsetzung: Frontend-Formulare auf `mitglied-werden.html` und `spenden.html` zeigen bei Fetch-Fehler eine User-freundliche Meldung („Antrag konnte nicht übermittelt werden – bitte später erneut versuchen.") statt eines Browser-Fehlers. Wenn der Reverse-Proxy keine Backend-Antwort bekommt, antwortet er mit `502 Bad Gateway`, was im JS-Fetch als Fehler ankommt.

6.4 Datenbank-Ausfall im Monitoring sichtbar

Umsetzung: Grafana-Alert-Regel **„Datenbank ausgefallen"** (Folder `DB`, Interval 10 s) auf `pg_up{job="postgres"}`. Bei `< 1` für ≥ 20 s feuert der Alert.

6.5 Frontend zeigt bei DB-Ausfall passende Fehlermeldung

Umsetzung: Backend `/health` antwortet bei DB-Verlust mit `503` (`db: "disconnected"`). Bei Schreib-Endpunkten (`POST /api/...`) wird der Insert-Fehler abgefangen und als `500 { error: "Interner Serverfehler" }` an das Frontend zurückgegeben, das daraufhin die Standard-Fehlermeldung anzeigt.

6.7 Automatischer Container-Restart

Umsetzung: Jeder Service hat `restart: unless-stopped`. Crasht ein Container, startet Docker ihn automatisch neu. Healthcheck-Failures zählen nicht als Crash, lösen also keinen Restart aus — das ist gewollt, weil Self-Healing-Loops bei z. B. einer hängenden DB sonst die Symptome verschleiern.

7. Monitoring & Alerting

7.1 Prometheus-Targets

Konfiguration: `max/monitoring/prometheus.yml`. Scrape-Interval: **15 s**.

Job	Targets	Was wird gemessen?
`cadvisor`	`cadvisor:8080`	CPU/RAM/Netzwerk pro Container
`nginx-exporter`	`nginx-exporter:9113`	Reverse-Proxy `stub_status` (active connections, requests/s)
`backend`	`backend-1:3000`, `backend-2:3000`	App-spezifische Metriken (Request-Histogram, Node-Defaults)
`prometheus`	`localhost:9090`	Selbst-Monitoring
`postgres`	`postgres-exporter:9187`	`pg_up`, Connections, Locks, Query-Counts
`blackbox`	HTTP-Probes auf `frontend`, `load-balancer`, `reverse-proxy`	End-to-End-Erreichbarkeit aus Sicht des Probers

Der Blackbox Exporter ist bewusst in `internal-net` und `public-net` eingebunden – `internal-net` damit Prometheus ihn scrapen kann, public-net damit er aktiv HTTP-Probes gegen die öffentlich erreichbaren Services durchführen kann.

7.2 Grafana-Dashboards

Auto-provisioniert aus `max/monitoring/grafana/provisioning/dashboards/dashboard.json`. Datenquelle (`prometheus`) ebenfalls über Provisioning gesetzt — kein manuelles Setup nach Grafana-Start nötig.

- Login: http://localhost:3001
- User `admin`
- Passwort aus `secrets/grafana_admin_password.txt`.

Bekannte Limitation: *cAdvisor liefert auf Docker Desktop für Mac keine `name`-Labels, weshalb die CPU/RAM-Panels dort leer bleiben. Auf Linux-Hosts funktioniert es. Workaround wäre ein anderer Container-Metrik-Sammler (z. B. `node-exporter` mit `--collector.cgroups`).*

7.3 Alert-Regeln

Definiert in `max/monitoring/grafana/provisioning/alerting/rules.yaml`. Alle Alerts gehen über die `policies.yaml`-Routing-Tabelle an den Empfänger `webhook-test`, der wiederum auf den Container `webhook-logger` zeigt (`http://webhook-logger:8080`).

Folder	Regel	Bedingung	Wartezeit
DB	Datenbank ausgefallen	`pg_up < 1`	20 s
Container	Container abgestürzt	`up{job=~"backend|postgres|prometheus|cadvisor|nginx-exporter"} < 1`	30 s
Container	CPU-Alert Applikation	`rate(container_cpu_usage_seconds_total{name=~"backend-1|backend-2|database"}[1m]) * 100 > 70`	1 min
Container	CPU-Alert Infrastruktur	dito für Nginx-/Exporter-Container, Schwelle `> 30 %`	1 min

Container	RAM-Alert	`container_memory_working_set_bytes / 1024 / 1024 > 800`	1 min
Backend	Backend ausgefallen	`up{job="backend"} < 1`	20 s
HTTP	HTTP-Service nicht erreichbar	`probe_success{job="blackbox"} < 1`	1 min

Warum unterschiedliche CPU-Schwellen?

App-Container (Backend, Datenbank) dürfen unter Last hochlaufen — 70 % sind erst dann auffällig, wenn es dauerhaft passiert. Infrastruktur-Container (Nginx, Exporter) sollten dauerhaft im einstelligen %-Bereich liegen; alles über 30 % deutet auf einen Bug oder ein DoS-Muster hin.

Notification Policy

Einstellung	Wert	Bedeutung
group_wait	10 s	Webhook kommt ~10 s nach erstem Feuern
group_interval	2 min	Resolved-Nachricht kommt max. 2 min nach Normalisierung
repeat_interval	4h	Wiederholung bei dauerhaftem Ausfall alle 4 h

Alerting-Verzögerung: Durch die mehrstufige Architektur vergehen zwischen einem tatsächlichen Ausfall und der Webhook-Nachricht im besten Fall ca. 1–2 Minuten:

Ausfall tritt auf
- ➔ Prometheus erkennt beim nächsten Scrape (max. 15 s)
- ➔ Grafana evaluiert beim nächsten Zyklus (max. 30 s)
- ➔ Pending Period abwarten (20 s – 1 min)
- ➔ group_wait abwarten (10 s)
- ➔ Webhook empfängt Alert

7.4 Verifikation eines Alerts

```
docker compose stop backend-1 #provoziert
"Backendausgefallen"-Alert
docker compose logs -f webhook-logger  #nach ~20-30 s erscheint der
POST mit JSON-Payload
docker compose start backend-1  Resolved-Nachricht folgt
```

8. Sicherheit

8.1 Netzwerkisolation

Siehe unter 2.2. Kernpunkt: `internal-net: internal: true` schneidet Backend und DB vom Internet ab.

8.2 TLS

- **Selbst-signiertes Zertifikat** unter `max/nginx/certs/`, gemountet `:ro` in den Reverse-Proxy.
- **TLS 1.2 und 1.3**, `HIGH:!aNULL:!MD5` als Cipher-Suite-Filter — schließt veraltete und null-cipher-Suiten aus.
- **HTTP→HTTPS Redirect 301** für alle Pfade außer `/rp-health` (Healthcheck soll ohne TLS-Handshake antworten).
- **HTTP/2 aktiv** (`http2 on`) — reduziert Latenz bei mehreren parallelen Requests pro Client.

8.3 Docker Secrets (statt ENV-Variablen)

Beide Passwörter (`POSTGRES_PASSWORD`, `GF_SECURITY_ADMIN_PASSWORD`) werden als **Docker Compose Secrets** definiert:

```
secrets:
  postgres_password:
    file: ./secrets/postgres_password.txt
```

- Im Container liegt das Secret unter `/run/secrets/<name>`.
- **Auswirkung von `docker inspect <container>`:** zeigt nur den Mount-Pfad, **nie den Inhalt**.

Wie die Services das Passwort einlesen:

Service	Mechanismus

database	Setzt POSTGRES_PASSWORD_FILE=/run/secrets/postgres_password — native Image-Funktion.
backend-1/2	Setzt dieselbe ENV; liest in server.js:10 mit fs.readFileSync.
grafana	Setzt GF_SECURITY_ADMIN_PASSWORD__FILE — native __FILE-Konvention von Grafana.
postgres-exporter	Setzt DATA_SOURCE_PASS_FILE.
db-backup	Liest im Entrypoint PGPASSWORD=$(cat /run/secrets/postgres_password).

8.4 Read-Only-Dateisysteme & Capabilities

Service	read_only	tmpfs	cap_drop	cap_add
database	—	/tmp, /var/run/postgresql	ALL	CHOWN, DAC_OVERRIDE, FOWNER, SETUID, SETGID
backend-1/2	✓	/tmp	ALL	—
load-balancer	✓	/tmp, /var/cache/nginx, /var/run	ALL	NET_BIND_SERVICE, CHOWN, SETUID, SETGID
frontend (-2)	✓	dito	ALL	dito
reverse-proxy	—	(per Image schreibbar)	ALL	dito

prometheus	✓	/tmp	ALL	—
grafana	✓	/tmp, /var/log/grafana	ALL	—
nginx-exporter	✓	/tmp	ALL	—
blackbox-exporter	✓	—	ALL	—
cadvisor	—	—	— (privileged)	—

Warum keine Read-Only-DB?

PostgreSQL schreibt bei `initdb` in mehrere System-Verzeichnisse außerhalb des Daten-Volumes. Der Erste Start würde mit Read-Only fehlschlagen. Stattdessen ist `cap_drop: ALL` plus eine Minimalliste von zugelassenen Capabilities die zweitbeste Härtung:

- `CHOWN`, `DAC_OVERRIDE`, `FOWNER`: Postgres legt sein Data-Dir mit eigenem User an.
- `SETUID`, `SETGID`: `gosu`-basierter Privileg-Wechsel root → `postgres`.

Warum Nginx-Container `NET_BIND_SERVICE` brauchen:

Nach `cap_drop: ALL` darf der Master-Prozess sonst keine Ports < 1024 mehr öffnen — auch in Container-Netzwerken. `CHOWN`/`SETUID`/`SETGID` braucht der Master, um Worker-Prozesse an den unprivilegierten `nginx`-User zu übergeben.

Warum cAdvisor `privileged: true`?

cAdvisor muss `/var/lib/docker`, cgroups und `/sys` lesen, um Container-Metriken zu sammeln — ohne `privileged` (oder eine sehr feingranulare Capability-Liste mit Host-Mounts) bricht das ab. Da cAdvisor selbst nur in das Monitoring-Netz hängt und keine eingehende Public-Verbindung hat, wird die Sicherheits-Reduktion akzeptiert.

8.5 Web Application Firewall (ModSecurity / OWASP CRS)

- **Image:** `owasp/modsecurity-crs:nginx-alpine`

- **Aktivierung:** `modsecurity on` + `modsecurity_rules_file /etc/modsecurity.d/setup.conf` in `max/nginx/reverse-proxy.conf`
- **Tuning:**
  - `PARANOIA=1` — Standard-Level (Level 4 würde viele harmlose Requests blocken).
  - `ANOMALY_INBOUND=5` — Score-Schwelle, ab der ein Request mit 403 abgewiesen wird.
  - `MODSEC_REQ_BODY_LIMIT=6291456` (6 MB) — Sonst würde die WAF die 5-MB-Lasttest-Bodies abweisen.

Funktionsnachweise (Auszug aus den Logs):

Angriff	Test-Request	WAF-Regel
SQL Injection	`?id=1' OR '1'='1`	`942100` „SQL Injection via libinjection"
Cross-Site Scripting	`?search=<script>alert('XSS')</script>`	`941100`/`941110` „XSS via libinjection"
Path Traversal	`?file=../../../../etc/passwd`	`930100`/`930120` „Path Traversal (/../)"

⇒ Alle drei Requests werden mit HTTP 403 Forbidden beantwortet, bevor sie das interne Netz erreichen.

8.6 Sicherheitsscan (Trivy)

Ein vollständiger Report des Trivy-Sicherheitscans ist im Projekt abgelegt unter: `max/security-scan/README.md`.

Image	Vorher	Nachher
`max-backend`	11 HIGH, 0 CRITICAL	**0 HIGH, 0 CRITICAL**
`max-frontend`	0 / 0	**0 / 0**
`max-database`	11 HIGH, 1 CRITICAL	**0 HIGH, 0 CRITICAL**

Maßnahmen:

1. **Backend Multi-Stage-Build** (`max/backend/Dockerfile`): Im Runtime-Stage werden `npm` und `npx` plus das gesamte `/usr/local/lib/node_modules/npm/`-Verzeichnis gelöscht. Die 11 HIGH-Befunde lagen ausschließlich in `npm`-Subdeps (`tar`, `glob`, `minimatch`, `cross-spawn`), die zur Laufzeit nicht gebraucht werden. `node server.js` braucht kein npm.
2. **Database Multi-Stage-Build** (`max/db/Dockerfile`): Das CRITICAL-Finding war **CVE-2025-68121** (`crypto/tls: incorrect certificate validation during TLS session resumption`) im `gosu`-Binary, das `postgres:17-alpine` mitbringt. Wurzel: gosu ist mit Go-stdlib v1.24.6 verlinkt; das Postgres-Team baut gosu nicht zeitnah neu. Lösung: gosu wird in Stage 1 aus dem aktuellen Quellcode mit `golang:1.25-alpine` (Go v1.25.10, CVE behoben) frisch kompiliert; Stage 2 überschreibt das verwundbare Binary unter `/usr/local/bin/gosu`.
3. **Frontend** brauchte keine Anpassung — `nginx:alpine` ist sauber.

8.7 Exponierte Ports

Port	Zweck	Begründung
`80`	HTTP → 301 Redirect auf 443 + Healthcheck	Browser-Standardport
`443`	HTTPS (alle App-Pfade, WAF, TLS)	Browser-Standardport
`3001`	Grafana	Bewertung erfordert Zugriff aufs Dashboard
`9999`	`webhook-logger` HTTP-Echo	Demo-Empfänger, in Produktion entfernen

Alle anderen Services sind **nicht** an den Host gebunden – Backend, DB, Prometheus, alle Exporter sind nur containerintern erreichbar.

9. Backup & Recovery

9.1 Aufbau

Sidecar-Container db-backup auf Basis von postgres:17-alpine:

```
db-backup:
  image: postgres:17-alpine
  secrets: [postgres_password]
  volumes:
    - db-backups:/backups
  networks: [internal-net]
  depends_on:
    database:
      condition: service_healthy
  entrypoint:
    - sh
    - -c
    - |
      while true; do
        PGPASSWORD=$$(cat /run/secrets/postgres_password) \
          pg_dump -h database -U ${POSTGRES_USER:-verein} ${POSTGRES_DB:-verein} \
          > /backups/backup_$$(date +%Y%m%d_%H%M%S).sql \
          && echo "Backup erfolgreich: $$(date)"
        sleep 3600
      done
```

Wesentliche Entscheidungen:

- **Eigener Container statt cron im DB-Container** — trennt Wartungslogik von der DB selbst und vermeidet `cron` in einem ohnehin schon zu schützenden Image.
- **`depends_on: condition: service_healthy`** — verhindert, dass das erste Backup gegen eine nicht-fertige DB läuft.
- **Passwort via Secret-File**, nicht ENV. `pg_dump` akzeptiert `PGPASSWORD` als ENV nur für die Dauer eines einzelnen Calls (Shell-Substitution `$( cat ... )`).
- **Eigenes Volume `db-backups`**, nicht im selben Volume wie `db-data`. Trennt Daten von Sicherungen und vermeidet, dass ein Bug der DB die Backups mitkorrumpiert.

9.2 Restore

```
# Verfügbare Backups listen
docker compose exec db-backup ls -lh /backups
# Restore eines bestimmten Snapshots
docker compose exec -T database psql -U verein -d verein \
  < <(docker compose exec db-backup cat
/backups/backup_20260531_120000.sql)
```

10. Lasttests

Ein vollständiger Report des K6-Lasttests ist im Projekt abgelegt unter: `max/loadtests/README.md`.

10.1 Setup

- **Tool:** k6, als Docker-Container, eingehängt im `max_public-net` — spricht den Reverse-Proxy unter `https://reverse-proxy/` an. Umgeht die TLS-Performance-Probleme von Docker Desktop unter macOS bei vielen parallelen Verbindungen.
- **Voraussetzung:** Docker-Desktop-Memory ≥ 6 GB (siehe Punkt 3.1).
- **Aufruf:** `./loadtests/run.sh <szenario>.js`.
- **Output:** `max/loadtests/results/`.

Es wurde sich hier für K6 entschieden, da auch das Monitoring in Grafana stattfindet und K6 ein bewährtes Tool aus dem Grafana-Kosmos ist.

10.2 Szenarien & Ergebnisse

#	Szenario	Kriterium	Req.	Fehler	p95	Notiz
01	web-10-parallel-0s	Z. 17	465 552	0 %	519 µs	31 k req/s
02	web-100-parallel-1s	Z. 18	426 088	0 %	7.28 ms	26 k req/s
03	web-1000-parallel-5s	Z. 19	7 500	0 %	870 µs	1000 req/s gehalten
04	web-1000-parallel-1s	Z. 20	5 500	0 %	822 µs	1000 req/s gehalten
05	web-1000rpm-10min	Z. 21	10 001	0 %	1.67 ms	16.67 req/s konstant
06	api-10-parallel-0s	Z. 23	125 370	0 %	1.89 ms	8.4 k req/s
07	api-100-parallel-1s	Z. 24	202 603	0 %	21.94 ms	9.6 k req/s

08	api-1000-parallel-5s	Z. 25	7 500	0 %	1.32 ms	1000 req/s gehalten
09	api-5mb-10-parallel-0s	Z. 26	2 020	0 %	75 ms	100 req/s, 500 MB/s
10	api-5mb-100-parallel-1s	Z. 27	1 491	0 %	3.68 s	47 req/s, 235 MB/s
11	api-5mb-1000-parallel-5s-429ok	Z. 28	737	0 × 5xx	5.62 s	0 × 429; 1573 k6-internal drops
12	api-5mb-1000-parallel-5s-strict	Z. 29	812	0 %	5.26 s	alle 812 mit 201

Alle 12 Szenarien laufen mit **0 Server-Fehlern** durch.

10.3 Nötige Anpassungen am Server-Code

Drei Stellen mussten verändert werden, damit 5-MB-Requests bis zur DB durchgehen:

- `backend/server.js`: `express.json({ limit: '6mb' })` — Default 100 KB, sonst HTTP 413.
- `nginx/reverse-proxy.conf`: `client_max_body_size 6m` — Default 1 MB, sonst HTTP 413 am Edge.
- `nginx/load-balancer.conf`: dito für den Load-Balancer.

10.4 Methodische Anmerkung

Für die 1000-VU-Szenarien (03, 04, 08, 11, 12) verwendet k6 den Executor ramping-arrival-rate statt ramping-vus. Grund:

- 1000 echte VUs brauchen in k6 ~6 GB RAM (jede VU hat eine eigene V8-Runtime). Das sprengt die Docker-Desktop-VM.
- `ramping-arrival-rate` definiert eine Ziel-Rate (z. B. 1000 req/s) und k6 nimmt nur so viele VUs wie nötig, um diese Rate zu halten (typisch 50–200).
- **Aus Server-Sicht** ist die Last identisch — der Reverse-Proxy sieht 1000 Requests pro Sekunde, unabhängig davon, ob sie aus 1000 oder 200 Client-Threads kommen.

Für die 5-MB-Szenarien ist die Rate auf 200 req/s reduziert, weil `5 MB × 1000 req/s = 5 GB/s` jede realistische Netzanbindung sprengt — die Tests würden Bandbreite messen, nicht Server-Verhalten.

10.5 Kurzbewertung

- **Statischer Web-Endpoint:** 30 k req/s mit Sub-Millisekunden-Latenz — komplett unproblematisch.
- **API-Endpoint (normale Bodies):** ~10 k req/s, p95 < 25 ms. Load-Balancer verteilt sauber auf beide Backend-Instanzen.
- **API-Endpoint (5 MB Bodies):** Limit ist die effektive Bandbreite, nicht der Server. p95 < 6 s, **keine 5xx-Fehler** in keinem Szenario. Damit sind sowohl Z. 28 (429 erlaubt) als auch Z. 29 (alle 201) erfüllt — das System weist schlicht keine Anfragen ab.

11. Bemerkungen

11.1 Bekannte Limitationen

cAdvisor auf Docker Desktop für Mac: liefert keine `name`-Labels für die Container, weshalb die CPU/RAM-Panels in Grafana auf dem Mac leer bleiben. Auf Linux-Hosts funktioniert es. Workaround wäre ein anderer Container-Metrik-Sammler.

11.2 Quellen & Werkzeuge

- Claude Code zur Unterstützung beim Herstellen einer Grundstruktur des Projekts und der Dokumentation. Bei der Erstellung des Codes von Frontend, Backend und DB (nicht infrastruktur-relevant) sowie Hilfe bei Verständnisfragen und Einschätzung von Fehlermeldungen.
- Docker Compose Specification (https://compose-spec.io/)
- OWASP ModSecurity CRS (https://coreruleset.org/)
- Trivy (https://github.com/aquasecurity/trivy)
- k6 Load Testing (https://k6.io/)
- Prometheus Documentation (https://prometheus.io/docs/)
- Grafana Provisioning (https://grafana.com/docs/grafana/latest/administration/provisioning/)
- CVE-2025-68121 (https://nvd.nist.gov/vuln/detail/CVE-2025-68121) – Go-stdlib crypto/tls- Bug, Grundlage für den gosu-Rebuild.